

Sr. No.	Contents	Page No.
Chapter 1	INTRODUCTION	
	1.1 Existing System	
	1.2 Problem Definition- Need of Computerization	
Chapter 2	PROPOSED SYSTEM	
	2.1 Proposed System	
	2.2 Objectives of System	
	2.3 User Requirements	
Chapter 3	ANALYSIS AND DESIGN	
	3.1 ERD, UML Diagram	
	3.2 Table Design	
	3.3 Data Dictionary	
	3.6 Test Procedures and Implementation	
Chapter 4	USER MANUAL	
	4.1 User Manual	
	4.2 Menu Explanation	
Chapter 5	CONCLUSIONS	
	5.1 Limitations & Drawbacks	
	5.2 Future Enhancement	
	5.3 Conclusions	
	5.4 References & Bibliography	

INTRODUCTION

The BFSI Fraud Detection and Analytics System is a data analytics and monitoring project developed to analyze banking transactions and identify fraudulent activities within financial systems. In the modern BFSI (Banking, Financial Services, and Insurance) sector, organizations process a huge number of transactions every day, making manual fraud monitoring difficult and inefficient. This project uses data warehousing concepts, SQL analytics, and business intelligence tools to store, process, and visualize transaction data effectively.

The system is designed using PostgreSQL as the database platform and Power BI for dashboard creation and reporting. Transaction data is stored in fact and dimension tables to support efficient querying and analytical processing. Various SQL queries are used to identify fraud trends, transaction patterns, fraud frequency, and total fraudulent transaction amounts.

The project helps in understanding how data analytics and visualization techniques can improve fraud monitoring and decision-making in financial institutions. By using dashboards and reports, users can easily monitor suspicious activities and generate meaningful business insights. The project also demonstrates practical implementation of database management, ETL concepts, and business intelligence tools in a real-world BFSI analytics scenario.

The project also demonstrates practical implementation of database management systems, ETL concepts, data warehousing architecture, SQL query optimization, and business intelligence tools commonly used in the BFSI industry. Through this implementation, the project provides hands-on exposure to fraud analytics workflows and highlights the importance of data-driven decision-making in modern financial systems.

Overall, the BFSI Fraud Detection and Analytics System serves as a foundational analytical solution for fraud monitoring and reporting. The project can be further enhanced by integrating real-time transaction processing, machine learning models, predictive fraud detection algorithms, and cloud-based analytics platforms to develop a more advanced enterprise-level fraud detection system.

1.1 Existing System

In the existing banking and financial environment, transaction monitoring and fraud detection are often handled using traditional database systems and manual reporting methods. Many organizations store transaction records in separate operational databases without implementing proper analytical processing or centralized data warehousing techniques. As a result, it becomes difficult to efficiently analyze large volumes of transaction data and identify suspicious or fraudulent activities in a timely manner.

Traditional systems mainly focus on transaction processing rather than fraud analytics and business intelligence reporting. Fraud identification is often performed manually by analyzing transaction logs and generating static reports, which is a time-consuming and inefficient process. These systems usually lack advanced analytical dashboards, centralized monitoring mechanisms, and visual reporting capabilities required for effective fraud analysis. Due to increasing digital transactions in the BFSI sector, manual monitoring approaches are no longer sufficient for detecting complex fraud patterns.

Existing systems also face challenges related to scalability and data integration. Transaction data generated from multiple banking channels such as online banking, credit cards, ATMs, and digital payment platforms is often stored in isolated systems, making consolidated analysis difficult. Without proper data warehouse architecture, organizations cannot efficiently perform trend analysis, historical analysis, or time-based fraud monitoring.

Additionally, existing systems generally do not support advanced business intelligence techniques such as KPI monitoring, trend visualization, drill-down reporting, and dimensional analysis. This reduces the ability of organizations to make fast and data-driven decisions related to fraud prevention and risk management.

Due to these limitations, there is a strong need for a computerized fraud detection and analytics system that can centralize transaction data, perform efficient analytical querying, and provide interactive dashboards for fraud monitoring. The proposed BFSI Fraud Detection and Analytics System addresses these issues by

1.2 Problem Definition – Need of Computerization

The rapid growth of digital banking, online payment systems, and electronic financial transactions has significantly increased the risk of fraudulent activities in the BFSI (Banking, Financial Services, and Insurance) sector. Financial institutions process thousands to millions of transactions daily, making manual fraud monitoring extremely difficult, time-consuming, and inefficient. Traditional transaction management systems are mainly designed for operational processing and do not provide advanced analytical capabilities required for fraud detection and monitoring.

In many existing systems, transaction data is stored across multiple databases and operational platforms without centralized analytical processing. Due to the absence of proper data warehousing and business intelligence mechanisms, organizations face challenges in identifying suspicious transaction behavior, monitoring fraud trends, and generating meaningful analytical reports. Static reports and manual analysis methods are not sufficient for handling large-scale transactional datasets generated in modern banking environments.

The need for computerization arises because financial organizations require:

- Centralized transaction data storage
- Faster fraud analysis and monitoring
- Automated SQL-based reporting
- Interactive analytical dashboards
- Efficient transaction trend analysis
- Better visualization of fraud patterns
- Improved decision-making using business intelligence tools

To address these challenges, the proposed BFSI Fraud Detection and Analytics System implements a computerized data warehousing and analytics solution using PostgreSQL, SQL, and Power BI. The system helps in storing transactional data in a structured warehouse schema, performing analytical fraud queries, and generating interactive dashboards for monitoring fraudulent activities. This computerized approach improves efficiency, reporting accuracy, and analytical capabilities in fraud monitoring processes.

PROPOSED SYSTEM

2.1 Proposed System

The proposed system is a **BFSI Fraud Detection and Analytics System** developed to analyze banking transaction data and identify potentially fraudulent activities using data warehousing, SQL analytics, and business intelligence techniques. The system is designed to provide a centralized platform for storing, processing, analyzing, and visualizing transaction-related information in an organized and efficient manner.

The proposed solution uses PostgreSQL as the backend relational database management system for storing transactional data and implementing warehouse schemas. Transaction data is loaded into staging tables and transformed into structured facts and dimension tables using ETL concepts. The warehouse schema follows a star schema model consisting of the `fact_transactions` table connected with dimension tables such as `dim_time`, `dim_customer`, and `dim_merchant`. This structure supports efficient querying, reporting, and analytical processing.

The proposed system provides several advantages over traditional transaction monitoring systems:

- Centralized transaction data management
- Faster analytical querying and reporting
- Improved fraud monitoring capabilities
- Interactive dashboards and visual analytics
- Better decision-making support
- Simplified transaction trend analysis
- Efficient fraud pattern visualization

The system is mainly intended for educational and analytical purposes to demonstrate the practical implementation of fraud analytics in the BFSI domain. It also provides hands-on exposure to database management, ETL processing, SQL querying, data warehousing, and Power BI dashboard development.

2.2 Objectives of System

The objectives of the project are:

- To build a centralized fraud monitoring database
- To analyze banking transaction data efficiently
- To identify fraudulent transactions using SQL queries
- To generate analytical reports and dashboards
- To visualize fraud patterns using Power BI
- To improve fraud monitoring and business intelligence reporting

2.3 User Requirements

The system requires the following user functionalities:

- Ability to store transaction data
- Ability to analyze fraud-related transactions
- Dashboard visualization for fraud analytics
- Report generation using SQL queries
- Easy monitoring of transaction patterns
- Fast retrieval of transaction information

2.4 Operating Environment – Hardware and Software

Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: 20 GB free disk space

Software Requirements

- Operating System: Windows 10/11
- Database: PostgreSQL
- Database Tool: pgAdmin 4
- Visualization Tool: Power BI Desktop
- Programming Language: SQL
- Optional Language: Python

ANALYSIS AND DESIGN

Module List

The project contains the following modules:

1. Data Loading Module

This module is used to import transaction datasets into the PostgreSQL database and load raw data into staging tables for processing.

Functions:

- ∄ Load transaction dataset
- ∄ Store raw transaction data
- ∄ Perform ETL-based data transformation
- ∄ Validate records before analysis

2. Data Warehouse Module

This module manages the warehouse schema and organizes transaction data into fact and dimension tables for analysis.

Functions:

- ∄ Create fact and dimension tables
- ∄ Manage warehouse schema
- ∄ Establish table relationships
- ∄ Support analytical querying

Main Tables:

- ∄ fact_transactions
- ∄ dim_time
- ∄ dim_customer
- ∄ dim_merchant

3. Transaction Analysis Module

This module performs SQL-based analysis on transaction data to generate transaction summaries and trends.

Functions:

- ✧ Analyze transaction counts
- ✧ Generate transaction summaries
- ✧ Perform time-based analysis
- ✧ Retrieve transaction insights

4. Fraud Detection Module

This module identifies and analyzes fraudulent transactions using analytical SQL queries.

Functions:

- ✧ Detect fraudulent transactions
- ✧ Analyze fraud counts
- ✧ Calculate fraud amounts
- ✧ Identify fraud trends

5. Dashboard and Reporting Module

This module creates dashboards and reports using Power BI for visualization and monitoring.

✧ Functions:

- ✧ Create interactive dashboards
- ✧ Display KPI indicators
- ✧ Generate charts and reports
- ✧ Visualize fraud patterns

✧ Dashboard Features:

- ✧ Total Transactions KPI
- ✧ Fraudulent Transactions KPI

- ∉ Total Fraud Amount KPI
- ∉ Fraud vs Non-Fraud Analysis
- ∉ Fraud Transactions by Hour

3.1 ERD, UML Diagram

ERD (Entity Relationship Diagram)

The database follows a star schema model containing:

Fact Table

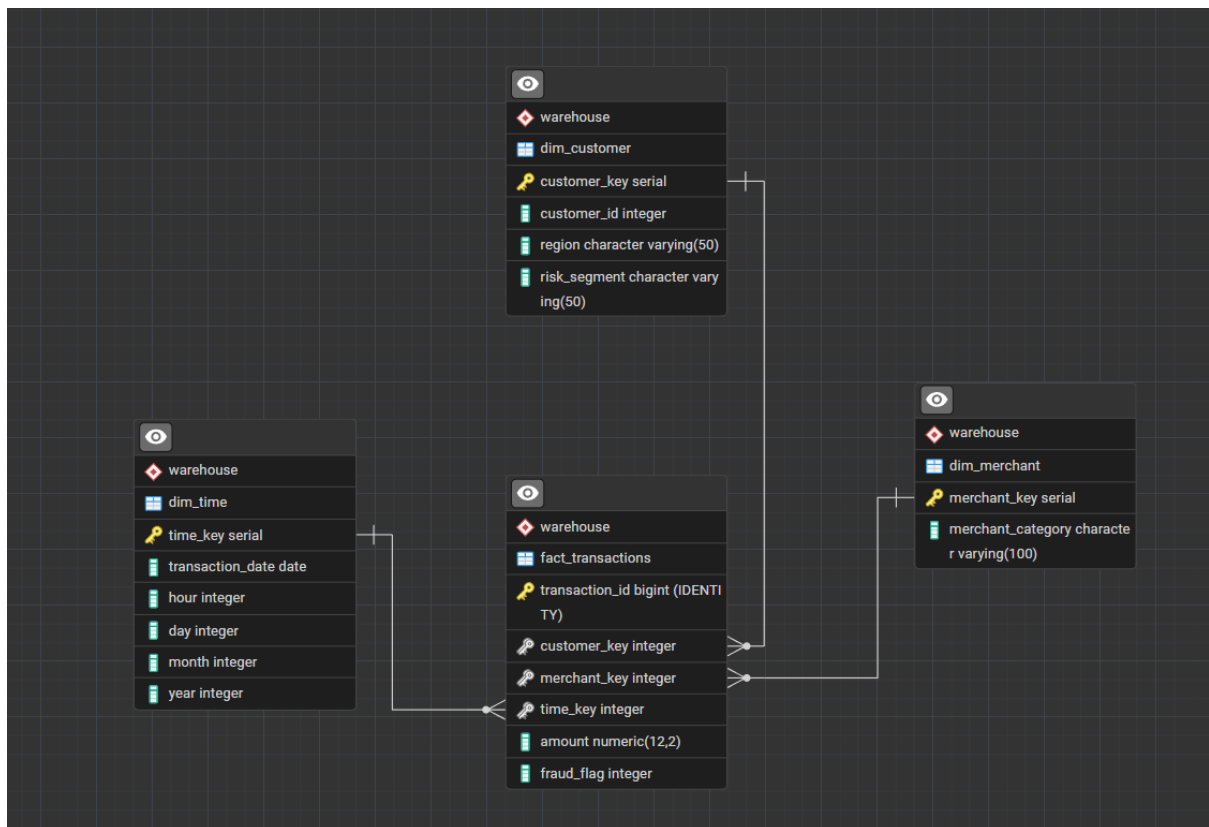
- fact_transactions

Dimension Tables

- dim_customer
- dim_merchant
- dim_time

Relationships

- customer_key connects fact_transactions with dim_customer
- merchant_key connects fact_transactions with dim_merchant
- time_key connects fact_transactions with dim_time



UML Diagrams:

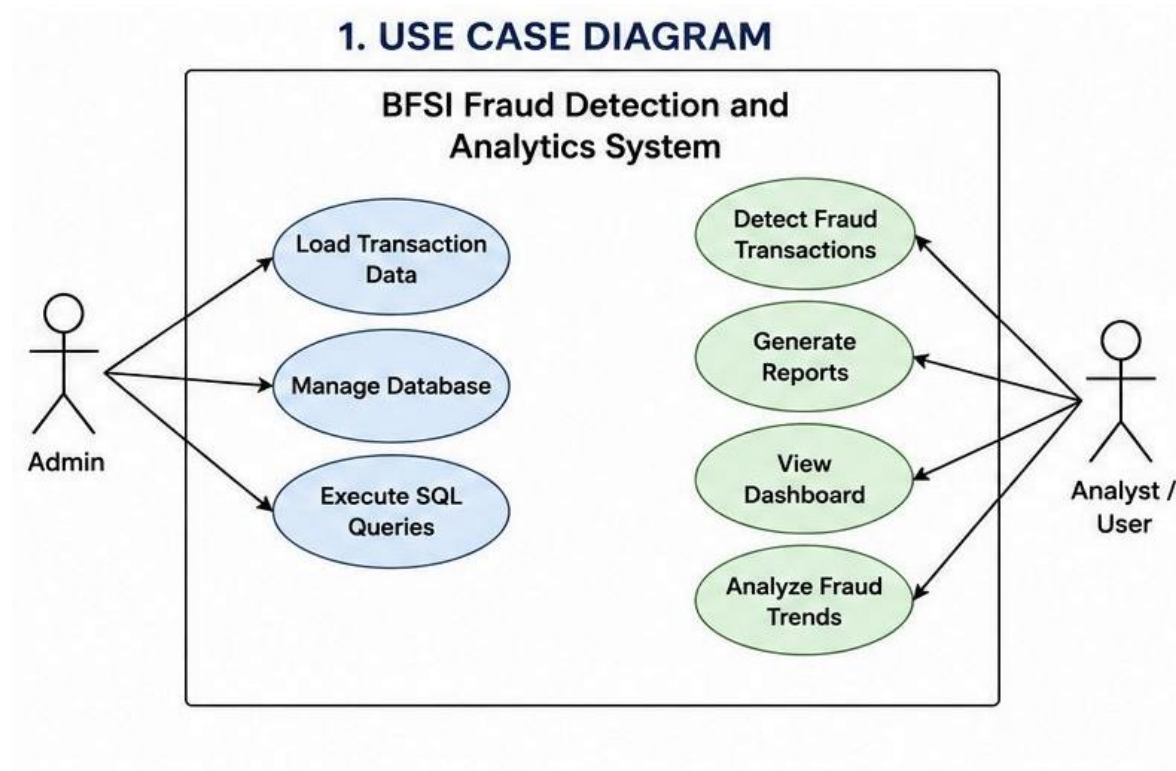
Use Case Diagram

Main Use Cases:

- Load transaction dataset
- Execute SQL queries
- Analyze fraudulent transactions
- Generate reports
- View Power BI dashboards
- Monitor fraud trends

Actors:

- Analyst/User
- Database System
- Power BI Dashboard



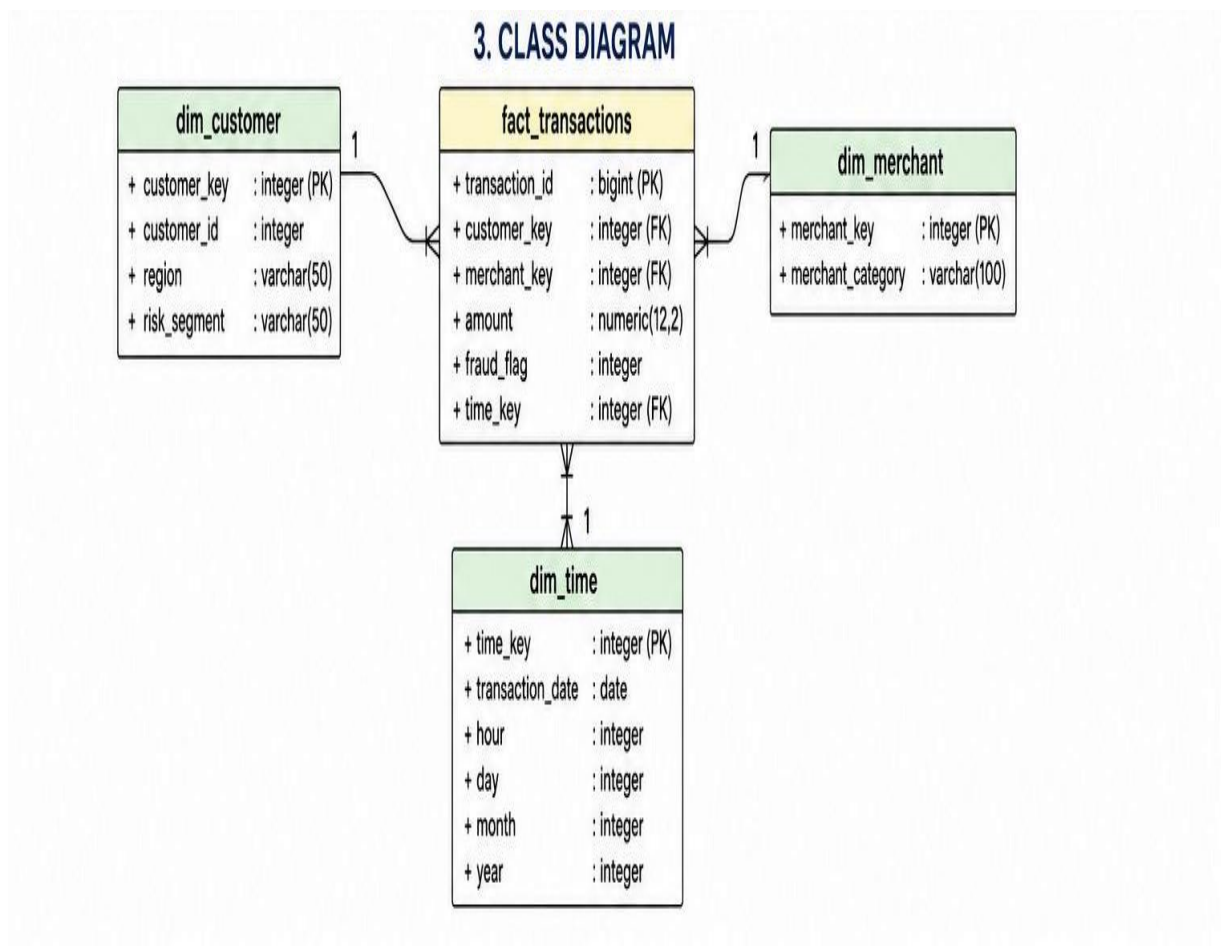
Class Diagram

Main Classes/Tables:

- fact_transactions
- dim_customer
- dim_merchant
- dim_time

Relationships:

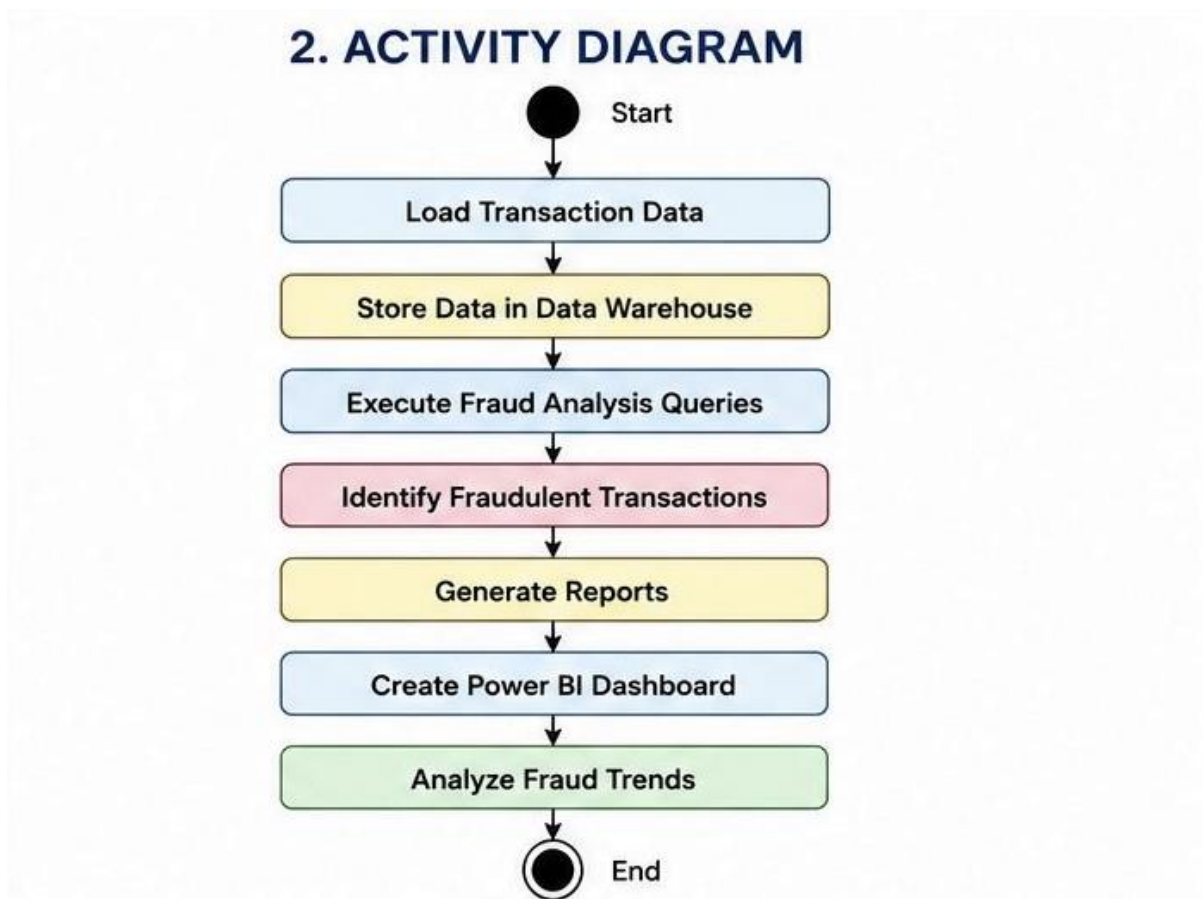
- fact_transactions connected with dimension tables using foreign keys



Activity Diagram:

Workflow:

1. Load transaction dataset
2. Store data in PostgreSQL
3. Create warehouse schema
4. Execute analytical SQL queries
5. Perform fraud analysis
6. Generate reports and dashboards
7. Visualize results in Power BI



3.2 Table Design

Table: fact_transactions

Column Name	Data Type	Description
transaction_id	Integer	Unique transaction ID
customer_key	Integer	Customer identifier
merchant_key	Integer	Merchant identifier
amount	Numeric	Transaction amount
fraud_flag	Integer	Fraud indicator
time_key	Integer	Foreign key from dim_time

Table: dim_time

Column Name	Data Type	Description
time_key	Integer	Primary key
transaction_date	Date	Transaction date
hour	Integer	Hour
day	Integer	Day
month	Integer	Month
year	Integer	Year

Table: dim_merchant

Column Name	Data Type	Description
merchant_key	SERIAL	Primary key
merchant_category	VARCHAR(100)	Merchant business category

Table: dim_customer

Column Name	Data Type	Description
customer_key	SERIAL	Primary key for customer dimension
customer_id	INTEGER	Original customer ID
region	VARCHAR(50)	Customer region/location
risk_segment	VARCHAR(20)	Customer risk category

3.3 Data Dictionary

The Data Dictionary defines the attributes and descriptions of the tables used in the BFSI Fraud Detection and Analytics System. It provides information about the purpose of each field within the warehouse schema.

fact_transactions

- transaction_id → Unique ID for every transaction
- customer_key → References customer dimension table
- merchant_key → References merchant dimension table
- time_key → References time dimension table
- amount → Transaction amount value
- fraud_flag → Indicates whether transaction is fraudulent (1) or normal (0)

dim_customer

- customer_key → Unique identifier for each customer
- customer_id → Original customer ID from dataset
- region → Customer geographic region
- risk_segment → Customer risk classification category

dim_merchant

- merchant_key → Unique identifier for each merchant
- merchant_category → Merchant business category

dim_time

- time_key → Unique identifier for time dimension

- transaction_date → Stores transaction date
- hour → Transaction hour
- day → Transaction day
- month → Transaction month
- year → Transaction year

3.4 Input and Output Screen Shots

Input Screens

- PostgreSQL table creation
- SQL query execution
- Data loading queries

	time_key [PK] integer	transaction_date date	hour integer	day integer	month integer	year integer
1	1	2000-01-01	0	1	1	2000
2	2	2000-01-01	1	1	1	2000
3	3	2000-01-01	2	1	1	2000
4	4	2000-01-01	3	1	1	2000
5	5	2000-01-01	4	1	1	2000
6	6	2000-01-01	5	1	1	2000
7	7	2000-01-01	6	1	1	2000
8	8	2000-01-01	7	1	1	2000
9	9	2000-01-01	8	1	1	2000
10	10	2000-01-01	9	1	1	2000
11	11	2000-01-01	10	1	1	2000
12	12	2000-01-01	11	1	1	2000
13	13	2000-01-01	12	1	1	2000
14	14	2000-01-02	0	2	1	2000
15	15	2000-01-02	1	2	1	2000
16	16	2000-01-02	2	2	1	2000
17	17	2000-01-02	3	2	1	2000
18	18	2000-01-02	4	2	1	2000
19	19	2000-01-02	5	2	1	2000
20	20	2000-01-02	6	2	1	2000
21	21	2000-01-02	7	2	1	2000
22	22	2000-01-02	8	2	1	2000

Total rows: 49 Query complete 00:00:00.129 CRLF Ln 1, Col 1

Output Screens

- Fraud transaction reports
- Fraud count analysis
- Power BI dashboards
- Charts and visualizations

3.5 Reports

The following reports were generated:

KPI cards were implemented in Power BI to display Total Transactions, Fraudulent Transactions, and Total Fraud Amount for quick analytical insights.

1. Fraud vs Non-Fraud Transactions

Query

Query History

1

SELECT fraud_flag, COUNT(*) AS total_transactions

2

FROM warehouse.fact_transactions

3

GROUP BY fraud_flag;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Showing rows: 1 to 2

	fraud_flag integer	total_transactions bigint
1	0	584033
2	1	1001

2. Fraud Transactions by Hour

Query

Query History

```
1 SELECT
2 d.hour,
3 COUNT(*) AS fraud_count
4 FROM warehouse.fact_transactions f
5 JOIN warehouse.dim_time d
6 ON f.time_key = d.time_key
7 WHERE fraud_flag = 1
8 GROUP BY d.hour
9 ORDER BY fraud_count DESC;
```

Data Output

Messages

Notifications

Showing rows: 1 to 24

Page No: 1 of 1

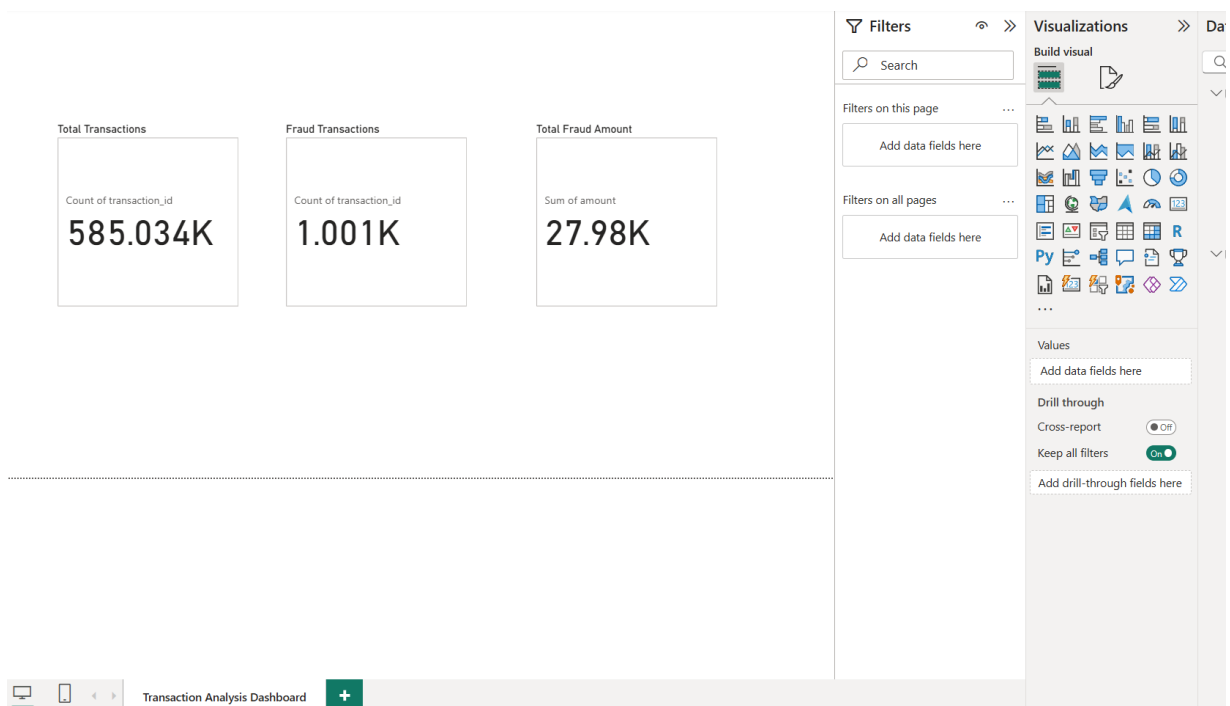
	hour integer	fraud_count bigint
10	16	44
11	23	42
12	19	38
13	20	36
14	3	34
15	13	34
16	9	32
17	21	32
18	5	22
19	1	20
20	22	18
21	6	18
22	8	18
23	10	16
24	0	12

3. Total Fraud Amount

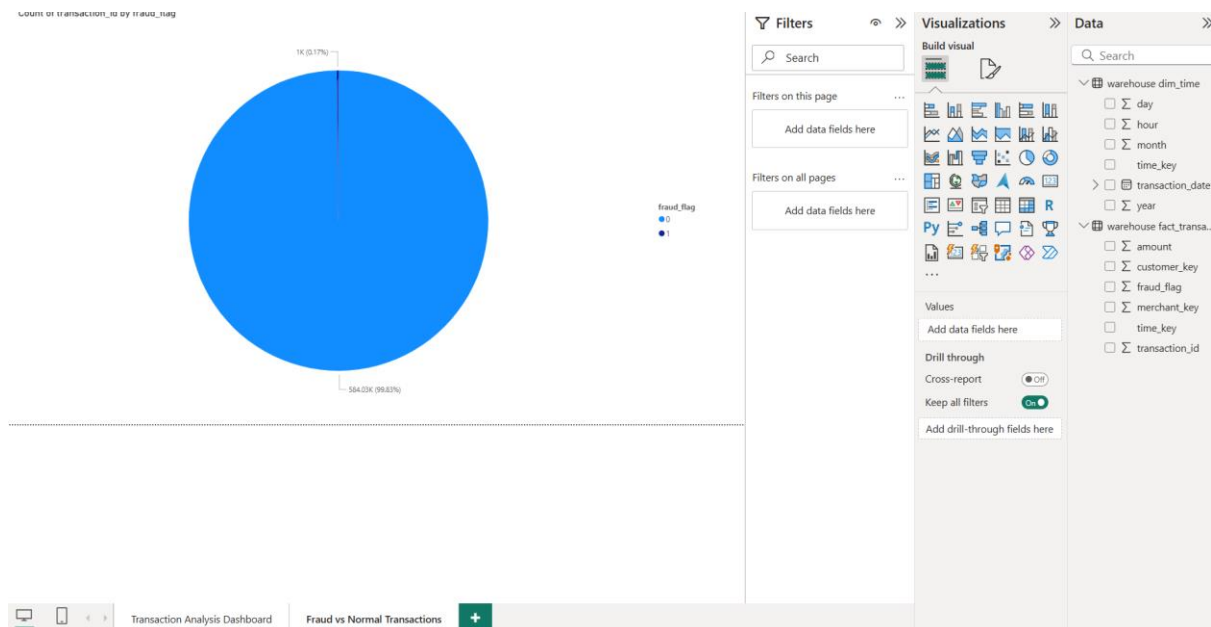
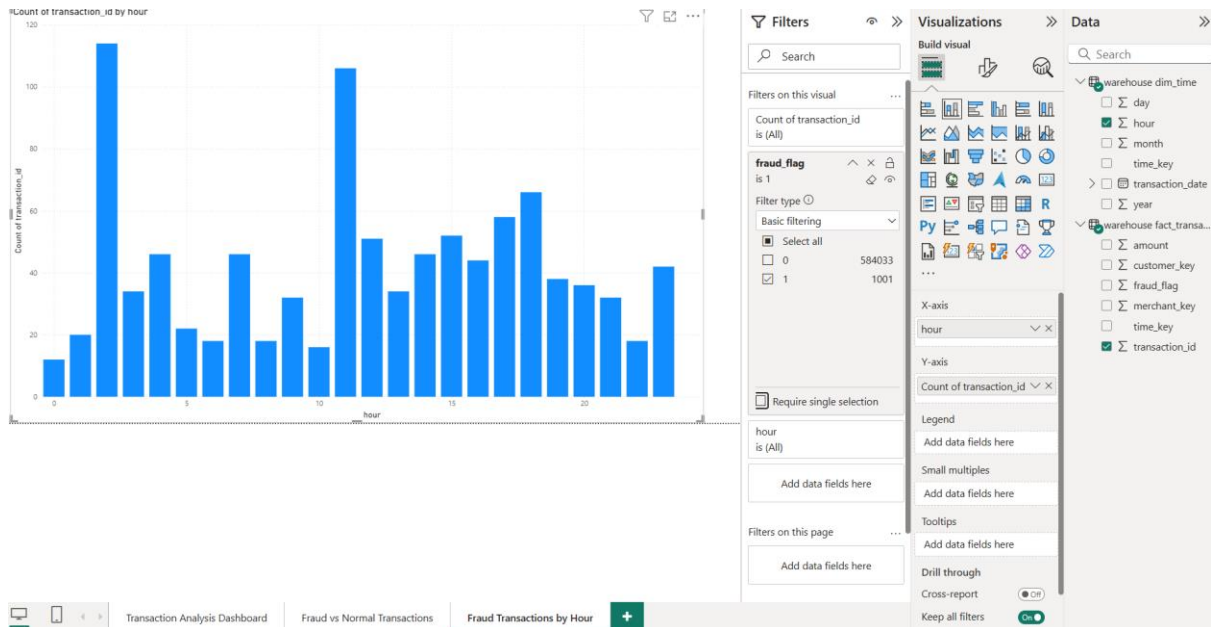
```
1 SELECT
2 SUM(amount) AS total_fraud_amount
3 FROM warehouse.fact_transactions
4 WHERE fraud_flag = 1;
```

Data Output		Messages	Notifications
Showing rows: 1 to 1 Page No: 1 of 1			
	total_fraud_amount		
	numeric		
1	123662.48		

4. Transaction Analysis Dashboard

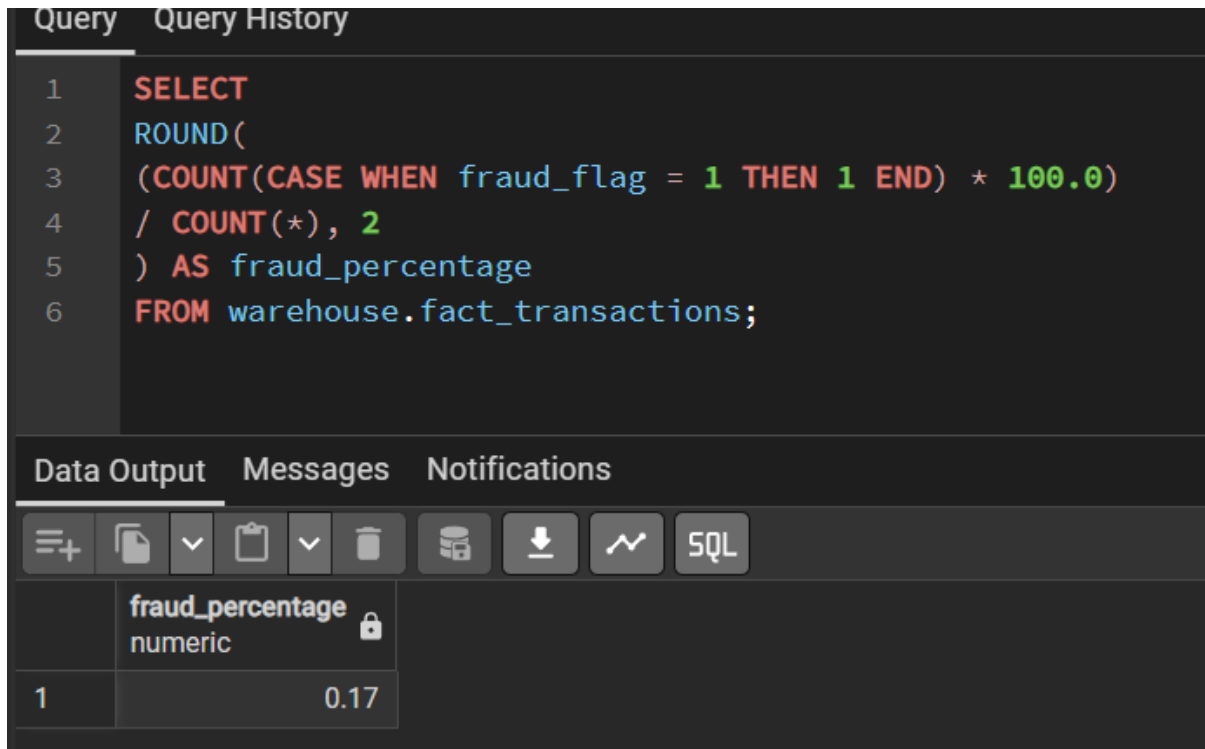


5. Fraud Trend Analysis



1.Fraud Percentage Analysis:

The query calculates the ratio of fraudulent transactions to total transactions and helps measure fraud occurrence within the transaction dataset.



The screenshot shows a SQL query editor with a dark theme. The query is as follows:

```
1 SELECT
2 ROUND(
3 (COUNT(CASE WHEN fraud_flag = 1 THEN 1 END) * 100.0)
4 / COUNT(*), 2
5 ) AS fraud_percentage
6 FROM warehouse.fact_transactions;
```

Below the query editor, there is a toolbar with icons for query execution, saving, and other functions. The results pane shows a table with one column, `fraud_percentage`, and one row with the value `0.17`.

	fraud_percentage numeric
1	0.17

2.Highest Fraud Transaction Amount

The query retrieves the maximum transaction amount associated with fraudulent activities. This helps identify high-value fraud cases and supports financial risk analysis.

3.Fraud Trend by Hour

Number of fraudulent transactions occurring during each hour

Query

Query History

1

2

3

4

5

6

7

8

9

SELECT

dt.hour,

COUNT(ft.transaction_id) AS fraud_transaction_count

FROM warehouse.fact_transactions ft

JOIN warehouse.dim_time dt

ON ft.time_key = dt.time_key

WHERE ft.fraud_flag = 1

GROUP BY dt.hour

ORDER BY dt.hour;

Data Output

Messages

Explain ×

Notifications

Showing rows: 1 to 24

Page No: 1

of 1

	hour integer	fraud_transaction_count bigint
1	0	12
2	1	20
3	2	114
4	3	34
5	4	46
6	5	22
7	6	18
8	7	46
9	8	18
10	9	32
11	10	16
12	11	106
13	12	51
14	13	34
15	14	46
16	15	52
17	16	44

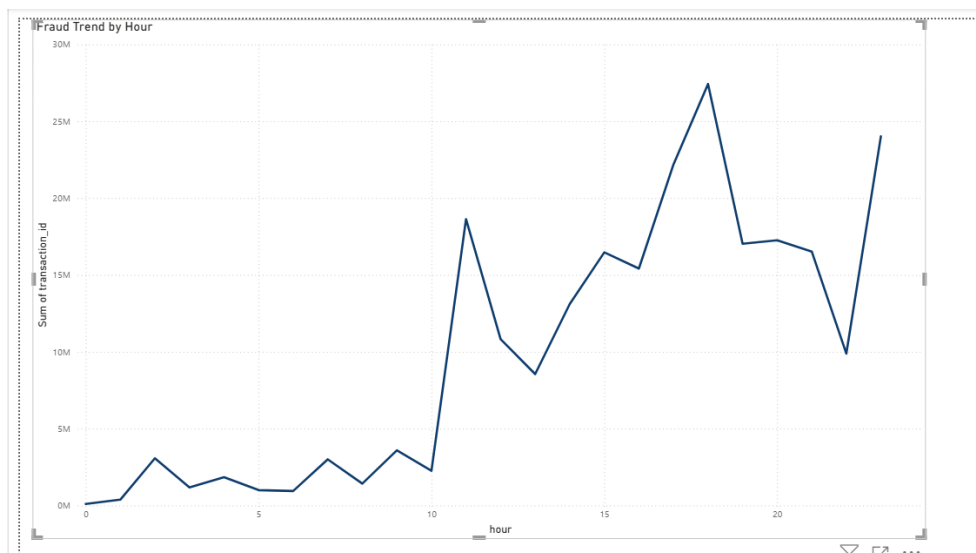
Total rows: 24

Query complete 00:00:00.182

CRLF

Ln 9, Col 18

Dashboard



3.6 Test Procedures and Implementation

Testing Performed

Test Case	Expected Result	Status
Database Connection	Successful connection	Passed
Data Loading	Data inserted successfully	Passed
Query Execution	Proper fraud analysis	Passed
Dashboard Loading	Charts displayed correctly	Passed

Implementation Steps

- Installed PostgreSQL and pgAdmin
- Created database and schemas
- Loaded transaction dataset
- Created fact and dimension tables
- Executed analytical SQL queries
- Connected database to Power BI
- Created fraud monitoring dashboard

USER MANUAL

4.1 User Manual

The BFSI Fraud Detection and Analytics System is designed to help users analyze banking transaction data and monitor fraudulent activities using PostgreSQL and Power BI dashboards. The system provides database management, SQL-based analysis, and interactive visualization features for fraud monitoring and reporting.

The following steps explain how to use the system:

Steps to Use the System

1. Open pgAdmin
2. Connect to PostgreSQL database
3. Open the fraud monitoring database
4. Execute SQL queries for analysis
5. Open Power BI dashboard
6. Refresh data for updated reports

4.2 Menu Explanation

pgAdmin Functions

- **Query Tool**

Used to execute SQL queries for transaction analysis and fraud monitoring.

- **Schemas**

Contains warehouse schema and database objects used in the project.

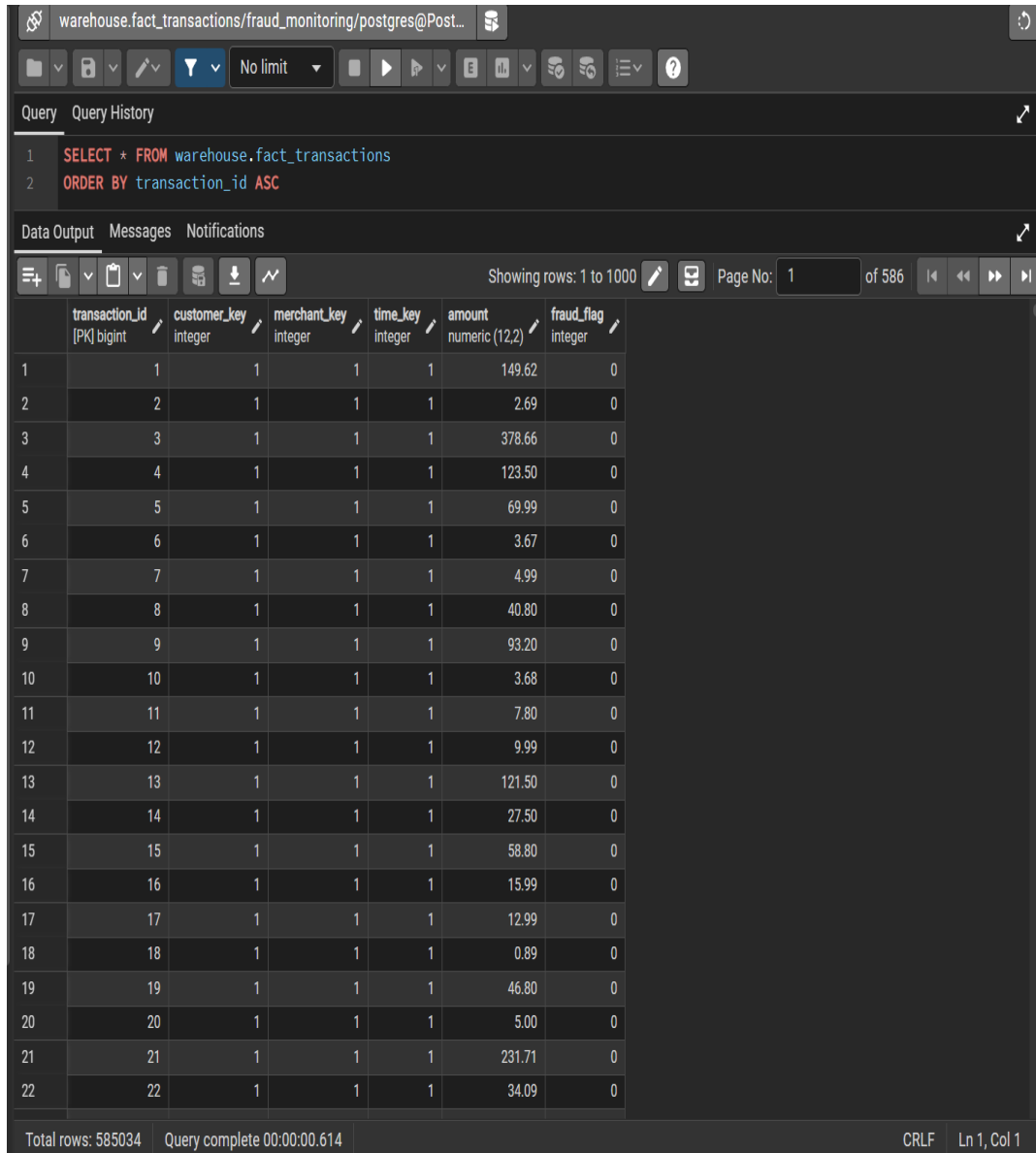
- **Tables**

Stores transaction records, dimension tables, and warehouse data.

- **ERD Tool**

Used to visualize table relationships and schema structure.

pgAdmin Interface



The screenshot shows the pgAdmin interface with a query window open. The query is: `SELECT * FROM warehouse.fact_transactions ORDER BY transaction_id ASC`. The results are displayed in a table with 8 columns: `transaction_id` (PK bigint), `customer_key` (integer), `merchant_key` (integer), `time_key` (integer), `amount` (numeric (12,2)), and `fraud_flag` (integer). The table shows 22 rows of data. The interface also includes a toolbar with various icons for query execution, a 'Query History' tab, and a 'Data Output' tab. The status bar at the bottom indicates 'Total rows: 585034' and 'Query complete 00:00:00.614'.

	transaction_id [PK] bigint	customer_key integer	merchant_key integer	time_key integer	amount numeric (12,2)	fraud_flag integer
1	1	1	1	1	149.62	0
2	2	1	1	1	2.69	0
3	3	1	1	1	378.66	0
4	4	1	1	1	123.50	0
5	5	1	1	1	69.99	0
6	6	1	1	1	3.67	0
7	7	1	1	1	4.99	0
8	8	1	1	1	40.80	0
9	9	1	1	1	93.20	0
10	10	1	1	1	3.68	0
11	11	1	1	1	7.80	0
12	12	1	1	1	9.99	0
13	13	1	1	1	121.50	0
14	14	1	1	1	27.50	0
15	15	1	1	1	58.80	0
16	16	1	1	1	15.99	0
17	17	1	1	1	12.99	0
18	18	1	1	1	0.89	0
19	19	1	1	1	46.80	0
20	20	1	1	1	5.00	0
21	21	1	1	1	231.71	0
22	22	1	1	1	34.09	0

Power BI Functions

- **Visualizations Pane**

Used to create charts, KPI cards, graphs, and dashboard visuals.

- **Filters Pane**

Used to filter transaction data and fraud-related records.

- **Data Pane**

Contains imported tables and fields connected from PostgreSQL.

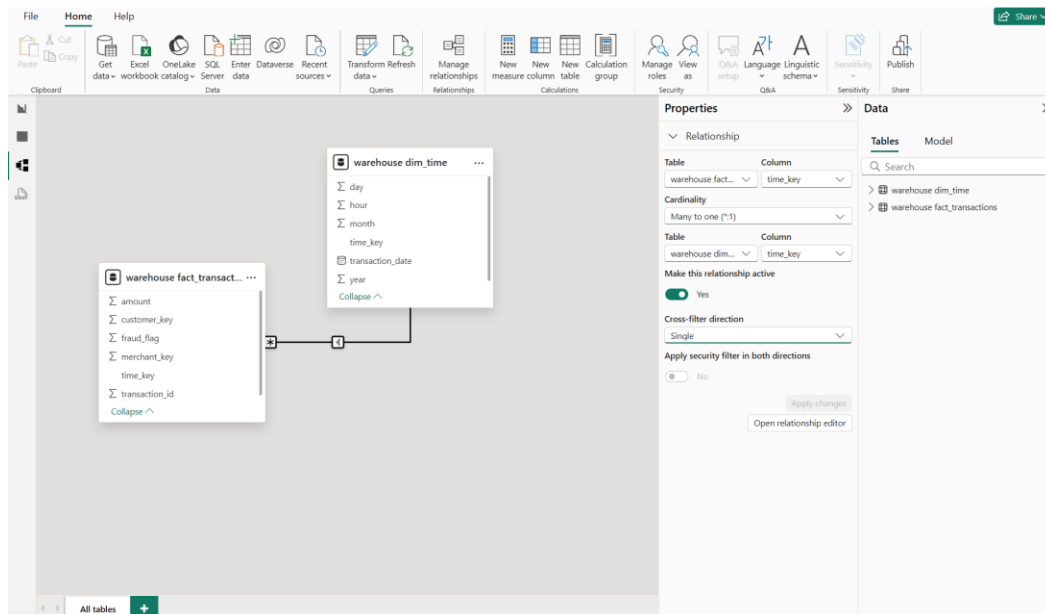
- **Report View**

Displays dashboards, charts, and analytical reports.

- **Refresh Option**

Updates Power BI dashboards with the latest database data

Power BI Dashboard



CONCLUSIONS

5.1 Limitations & Drawbacks

Although the Fraud Detection and Transaction Monitoring System provides useful analytical insights, the project has certain limitations. The system currently works on a static dataset and does not process real-time transaction streams. Due to limited project scope and internship duration, advanced machine learning algorithms for predictive fraud detection were not implemented. The dashboard primarily focuses on historical transaction analysis rather than automated fraud prevention.

The fraud analysis is performed using SQL-based querying techniques, and advanced machine learning algorithms for predictive fraud detection were not implemented. As a result, the system can identify and analyze fraudulent transactions present in the dataset but cannot automatically predict future fraud behavior or detect complex anomaly patterns in real time.

Another limitation is that the dataset used in the project contains limited customer and merchant attributes. Features such as transaction location, device information, customer behavioral history, and payment gateway details were not available, which restricts deeper fraud pattern analysis. The Power BI dashboards mainly provide historical analysis and visualization rather than automated alert generation or notification systems.

The project also depends on manual data loading and dashboard refresh operations. Large-scale performance optimization techniques such as indexing strategies, partitioning, and distributed processing were not implemented extensively. Since the project runs in a local PostgreSQL and Power BI Desktop environment, cloud deployment and multi-user accessibility are not included in the current version.

Security-related features such as role-based authentication, encrypted transaction storage, and API integration were also not implemented because the project scope was focused mainly on data warehousing, SQL analytics, and business intelligence reporting.

5.2 **Future Enhancement**

The BFSI Fraud Detection and Analytics System can be enhanced further by integrating advanced fraud detection and real-time analytical capabilities. Future versions of the project can include machine learning and artificial intelligence techniques to improve fraud prediction accuracy and automate suspicious transaction detection.

Advanced machine learning algorithms such as Logistic Regression, Decision Trees, Random Forest, and Neural Networks can be integrated to identify hidden fraud patterns and predict high-risk transactions automatically. Real-time transaction monitoring can also be implemented using streaming technologies and automated data pipelines for continuous fraud analysis.

The system can be further improved by integrating cloud-based platforms such as Microsoft Azure, AWS, or Google Cloud for scalable data storage and processing. This would help the system handle large-scale transactional datasets generated in enterprise BFSI environments. Additional optimization techniques such as indexing, partitioning, and query optimization can also improve analytical performance.

Future enhancements may also include:

- Real-time fraud alert generation
- Automated email or SMS notifications
- Role-based authentication and security
- Customer risk scoring system
- Geo-location-based fraud analysis
- Mobile dashboard accessibility
- API integration with banking systems
- Interactive drill-through analytics in Power BI

Overall, the project provides a strong foundation for developing a more advanced enterprise-level fraud monitoring and business intelligence system in the BFSI sector.

5.3 Conclusions

The BFSI Fraud Detection and Analytics System was successfully designed and implemented using PostgreSQL, pgAdmin, SQL, and Power BI. The project demonstrated the practical implementation of data warehousing concepts, database management, SQL-based analytical querying, and business intelligence reporting within the BFSI domain.

The system effectively stores transaction data in a structured warehouse schema consisting of fact and dimension tables. Analytical SQL queries were used to perform fraud analysis, transaction trend monitoring, fraud amount calculation, and time-based transaction analysis. The generated outputs were further visualized using interactive Power BI dashboards and KPI indicators for better analytical reporting and decision-making.

The project provided hands-on exposure to ETL concepts, warehouse schema design, relational database management, SQL query execution, and dashboard development techniques commonly used in industry-level analytics environments. Interactive dashboards such as Fraud Trend Analysis, Fraud vs Non-Fraud Transactions, and Transaction Analysis Dashboard helped transform raw transaction data into meaningful business insights.

The implemented system improves fraud monitoring efficiency by providing centralized transaction storage, faster analytical querying, and visual representation of fraud-related information. Through this project, the importance of business intelligence tools and data analytics techniques in financial fraud monitoring was clearly demonstrated.

Overall, the project serves as a strong analytical and reporting solution for fraud monitoring in the BFSI sector and provides a solid foundation for future enhancements such as real-time analytics, cloud integration, and machine learning-based fraud prediction systems.

References & Bibliography

Books Referenced

- Silberschatz, Korth & Sudarshan – *Database System Concepts*
- Ralph Kimball – *The Data Warehouse Toolkit*
- Microsoft Documentation – *Power BI Learning Guide*
- SQL Fundamentals and Query Reference Guides
- Banking and Financial Fraud Analytics Research Papers

Online References

- PostgreSQL Official Documentation
- <https://www.postgresql.org/docs/>
- pgAdmin Official Website
- <https://www.pgadmin.org/>
- Microsoft Power BI Documentation
- <https://learn.microsoft.com/en-us/power-bi/>
- Kaggle Dataset Platform
- <https://www.kaggle.com/>
- SQL Tutorial by W3Schools
- <https://www.w3schools.com/sql/>
- GeeksforGeeks DBMS Concepts
- <https://www.geeksforgeeks.org/dbms/>
- Star Schema Explanation
- <https://www.techtarget.com/searchdatamanagement/definition/star-schema>
- Power BI Visualization Tutorials
- <https://learn.microsoft.com/en-us/training/powerplatform/power-bi/>
- PostgreSQL Community Technical Resources
- <https://www.postgresql.org/community/>
- BFSI Analytics and Fraud Detection Case Studies
- <https://www.ibm.com/topics/fraud-detection>

